Unit 8 e-Portfolio Activity: Gradient Cost Function

Introduction

I ran the supplied tutorial, then tested learning rates, input scaling and a target that no line fits.

Method

The tutorial uses gradient descent (Cauchy, 1847, cited in Goodfellow, Bengio and Courville, 2016) to fit $y = mx + b$ to five points on $y = 2x + 3$, from $m = b = 0$. The derivatives of the mean squared error give each step's direction and the learning rate its size (University of Essex Online, 2026).

Findings

Of the rates I tried, 0.082 needed the fewest iterations, 245 against 251 at the supplied 0.08; 0.084 needed 734, and from 0.085 every run diverged (Tables 1 and 2). The most curved direction of the cost caps the learning rate, leaving slow progress along flatter ones (Goodfellow, Bengio and Courville, 2016). Here it was 70 times as curved as the flattest, and the update rule allows only rates below 0.0845. At 0.08 each step overshot the minimum, as Mayo (2017) describes, and the oscillation decayed more slowly than at 0.05 (Table 2), leaving a higher cost after 10 iterations. The cost is convex, with one global minimum (Géron, 2025); 0.085 diverged by exceeding the limit, not by missing a minimum, the risk the supplied notebook names. Standardising x, as Géron (2025) recommends, raised the limit to 1 and cut the iterations at 0.08 from 251 to 53 (Table 3). With the last target changed from 13 to 18, the stable runs settled on $y = 3x + 1$ at a cost of 2, while 0.085 still diverged (Table 4).

Implications and conclusion

The inputs set the safe learning rate; the targets set the lowest reachable cost, so a cost stuck above zero may mean that no line fits, not that the optimiser failed. I used only constant rates, though Géron (2025) argues that a schedule lowering the rate during training can do better. A lower training cost does not show performance on unseen data (Goodfellow, Bengio and Courville, 2016).

Changing the iterations and the learning rate

Learning rate	10 iterations	100 iterations	1,000 iterations	10,000 iterations
0.001	55.5	1.61	0.478	0.00109
0.01	1.28	0.477	0.00107	1.04×10^{-27}
0.05	0.668	0.031	1.46×10^{-15}	3.17×10^{-29}
0.08 (supplied)	9.69	0.0039 (supplied run)	1.43×10^{-24}	1.03×10^{-29}
0.085	Diverged	Diverged	Diverged	Diverged
0.09	Diverged	Diverged	Diverged	Diverged

Table 1: Cost (mean squared error) after the final update. The cost at the start is 89; ‘diverged’ means it ended higher than that. Costs this small mean the line matches y = 2x + 3 to many decimal places.

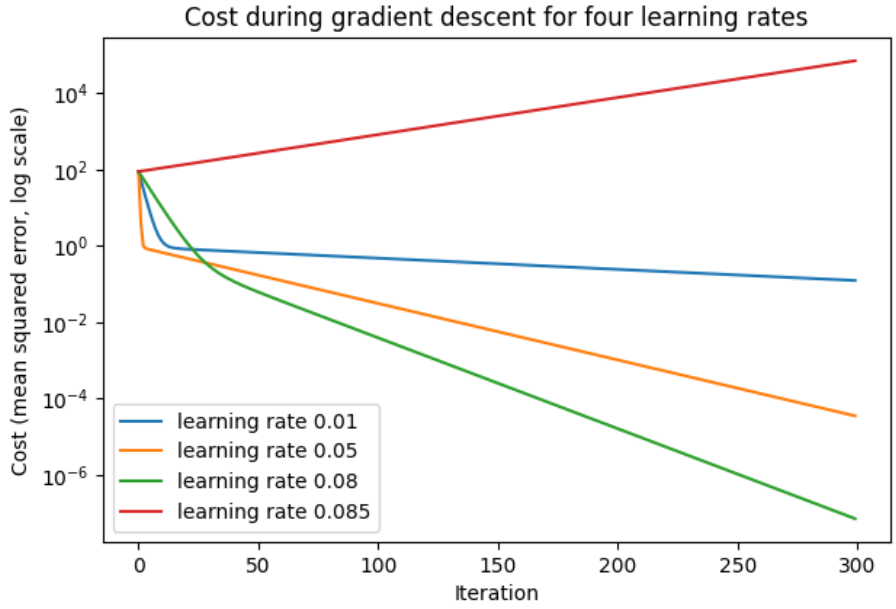


Figure 1: Cost during the first 300 iterations for four learning rates, on a log scale.

Where the learning rate becomes too large

Learning rate	Iterations to a cost below 0.000001	Factor per step, steepest direction	Outcome
0.001	20,336	0.976	Converged
0.01	2,031	0.763	Converged
0.05	404	−0.183	Converged
0.07	288	−0.656	Converged
0.08 (supplied)	251	−0.893	Converged
0.081	248	−0.917	Converged
0.082	245	−0.940	Converged
0.083	257	−0.964	Converged
0.084	734	−0.988	Converged
0.085	Never (diverged)	−1.011	Diverged

Table 2: The threshold of 0.000001 is my own choice. From the update rule, each step multiplies the remaining distance to the minimum along each of the bowl's two main directions by a factor: 1 minus the learning rate times that direction's curvature. A negative factor means the step overshoots; a size above 1 means the steps grow. The steps therefore shrink only for learning rates below 2 divided by the largest eigenvalue of the cost's Hessian matrix: 2 / 23.662, about 0.0845. The scan from 0.080 to 0.090 converged up to 0.084 and diverged from 0.085.

Measure	x as supplied	x standardised
Curvature of the cost (eigenvalues)	0.338 and 23.662	2 and 2
Learning rates whose steps shrink	Below 0.0845	Below 1
Iterations to a cost below 0.000001, learning rate 0.01	2,031	454
Same, learning rate 0.05	404	87
Same, learning rate 0.08	251	53
Same, learning rate 0.5	Not run (above 0.0845)	1
Same, learning rate 0.9	Not run (above 0.0845)	42

Table 3: The same runs with x standardised. Converted back, the scaled run gives the same line, m = 2 and b = 3. A learning rate of 0.5 reaches the minimum in one step because its factor per step, 1 − 0.5 × 2, is zero.

Learning rate, 1,000 iterations	m	b	Cost
0.05	3.0000	1.0000	2.0000
0.08	3.0000	1.0000	2.0000
0.084	3.0000	1.0000	2.0000
0.085	Diverged	Diverged	Diverged

Table 4: The last target changed from 13 to 18. The least-squares line is $y = 3x + 1$, with a lowest possible cost of 2. The limit is unchanged because the second derivatives of the cost involve only x.

Changes to the supplied notebook

Type	In the supplied notebook	Change
Correction	Each printed line paired the updated m and b with the cost from before the update.	The cost is printed beside the m and b that produced it.
Change for the experiments	The iterations and learning rate were fixed inside the function, which returned nothing.	Both are arguments, with the supplied 100 and 0.08 as defaults; the function returns m, b and the cost at every step.
Housekeeping	An empty code cell at the end.	Removed.

Table 5: The changes, also listed in the notebook's opening cell.

References

Géron, A. (2025) *Hands-On Machine Learning with Scikit-Learn and PyTorch*. Santa Rosa: O'Reilly.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press. Available at: https://www.deeplearningbook.org/ (Accessed: 3 November 2026).

Mayo, M. (2017) 'Neural network foundations, explained: updating weights with gradient descent & backpropagation', *KDnuggets*, 25 October. Available at: https://www.kdnuggets.com/2017/10/neural-network-foundations-explained-gradient-descent.html (Accessed: 20 October 2026).

University of Essex Online (2026) 'Unit 8 Lecturecast: Backpropagation in Artificial Neural Networks' [Lecturecast]. ML_PCOM7E: Machine Learning October 2026. University of Essex Online. Available at: https://www.my-course.co.uk/pluginfile.php/1659724/mod_scorm/content/2/scormcontent/index.html (Accessed: 30 October 2026).

Supplied materials: University of Essex Unit 8 e-Portfolio brief and the notebook gradient_descent_cost_function.ipynb (code credit: codebasics).

AI Acknowledgement

Codex was used to assist in identifying appropriate code solutions and to proofread and refine the code I produced. Claude and Grammarly supported proofreading, language refinement, and formatting of my written text. All final decisions, analyses, interpretations, conclusions, and recommendations are my own.